

Tech stack overview

The requirements appear to be satisfiable by a fairly standard CRUD webapp, with perhaps the only interesting parts being:

1. Usable image annotation UI on the frontend
2. Potentially large blob storage for uploaded images

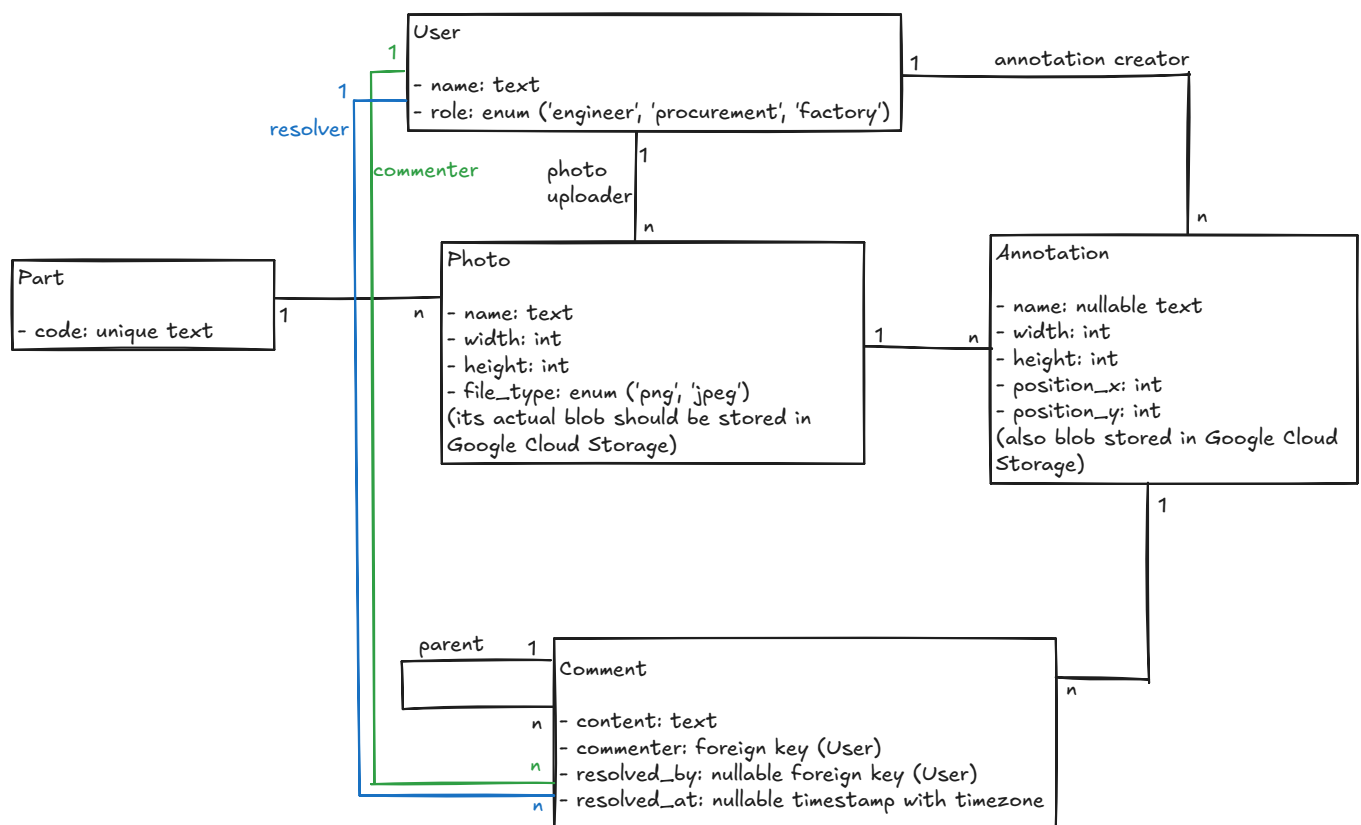
To stay consistent with CADDi's existing standards, we'll use the same full-stack TypeScript approach:



I'm assuming CADDi already has reusable building blocks for such a setup (boilerplate FE/BFF/BE repos, terraform code for multiple dev/staging/prod environments, React component library that follows company-wide design system, etc). If any of these are absent, it might be a good idea to start building them.

Database

A GCP-managed Postgres should suffice. Here's a rough sketch of the DB schema:



A comment can be either a thread starter (parent is null) or a reply to another comment (parent is not null)

Common columns not shown in diagram for brevity:

- Every table has a 64-bit integer primary key named `id`
- Most tables should have a `created_at` column that's a timestamp with timezone, to avoid any ambiguity.

I'm assuming each comment thread can be eventually resolved, to avoid visual clutter over time.

For the first MVP, we'll allow only 1 type of annotation: a "brush" that allows the user to freely draw on top of a photo, with adjustable line width and color. Once they're done, we simply store what's drawn as a separate image. The user can then comment on their newly created annotation.

Since the customer is a 500+ employee company, we'll most likely need to implement user tagging inside each comment, so that we can notify relevant employees via email, or a dashboard, etc.

Authentication

Use Auth0 just like CADDi's standard stack.

I assume, since the customer company size is 500+, that they already use a corporate identity provider (IdP): Okta, Microsoft, Google, etc. - naturally, we'll need to configure our system to work with their IdP of choice. I deliberately left out things like user passwords in the DB schema above because user management should be the IdP's responsibility.

Frontend

Since the use case involves taking photos, our software should be usable on smartphones and tablets, not just computers. I personally prefer native mobile apps, but since I don't see any mention of Android/iOS in CADDi's company overview material, I assume we currently don't have native mobile developers. Therefore, let's make a mobile-friendly single-page webapp for the MVP. In the long run, I still think native mobile apps offer the best user experience, so we may want to start hiring for native mobile dev.

Runs on GKE Pods. Put behind a CDN (Cloudflare) for simple protection against malicious traffic.

Backend

Runs on GKE Pods.

Monitoring

Use cloud-hosted solutions:

- Datadog for performance monitoring and tracing. Setup alerts via chat/email if performance indicators (e.g. p95 API response time) drop below acceptable baseline.
- Sentry for runtime error detection and triage.

CI/CD

- All automated tests must pass. No warning allowed.
- Most (or if possible, all) code conventions should also be checked by CI. Use opinionated tools like `prettier` to avoid pointless arguments in code review.

Testing

- Automated: use NestJS's provided testing facilities
- Manual: provide multiple live environments. Exactly how many environments are needed depends on the company's development & QA process.